In [25]:
```python
import json
import os
```

In [26]:
```python
# ==================== 1. 加载结果文件 ====================
RESULTS_PATH = "../Output/results_final.json"
COMPARISON_PATH = "../Output/comparison_final.txt"

# 检查文件是否存在
if not os.path.exists(RESULTS_PATH):
    raise FileNotFoundError(f"Not found: {RESULTS_PATH}\n")
if not os.path.exists(COMPARISON_PATH):
    raise FileNotFoundError(f"Not found: {COMPARISON_PATH}")

# 读取 JSON 结果
with open(RESULTS_PATH, "r", encoding="utf-8") as f:
    results = json.load(f)

# 读取对比文本
with open(COMPARISON_PATH, "r", encoding="utf-8") as f:
    comparison_text = f.read()
```

In [27]:
```python
# ==================== 2. 展示测试精度 ====================
print("=" * 60)
print("Accuracy of four methods on IMDB test set")
print("=" * 60)
accuracies = results["test_accuracies"]
print(f"Method 1 (One-Hot Bag of Words + MLP) : {accuracies['method1']*100:.2f}%")
print(f"Method 2 (Word2Vec + BiGRU) : {accuracies['method2']*100:.2f}%")
print(f"Method 3 (Frozen DistilBERT + Logistic Regression): {accuracies['method3']*100:.2f}%")
print(f"Method 4 (Few-shot, 20-shot, 10 trials) : {accuracies['method4']*100:.2f}%")
print()
```

```
============================================================
Accuracy of four methods on IMDB test set
============================================================
Method 1 (One-Hot Bag of Words + MLP) : 85.05%
Method 2 (Word2Vec + BiGRU) : 90.85%
Method 3 (Frozen DistilBERT + Logistic Regression): 84.28%
Method 4 (Few-shot, 20-shot, 10 trials) : 66.57%
```

3. 参数量对比分析（One-Hot vs Word2Vec）

对比项	**One-Hot + MLP**	**Word2Vec + BiGRU**
词表大小	20,000	51,955
嵌入/第一层维度	1024	300
第一层/嵌入参数量	**20,480,000(approx.)**	**15,586,500(approx.)**
模型总参数量	21,141,250	17,628,234
测试准确率	85.05%	**90.85%**

结论

- One-Hot 虽限制词表（20k），但第一层参数量（20.48M）仍 **比** Word2Vec 嵌入矩阵（15.59M）多 **31%**
- Word2Vec 参数量更少、精度更高，体现分布式表示的优越性

四种方法对比分析

方法	词嵌入方式	**模型**	测试精度	**参数量**	**优点**	**缺点**
方法1	One-Hot 词袋	MLP	**85.05%**	21.1M	实现简单，无需预训练	维度高，丢失语义，必须限制词表
方法2	Word2Vec	BiGRU	**90.85%**	17.6M	捕获语义和词序，参数量更少	需要训练词向量，无法处理未登录词

方法	词嵌入方式	模型	测试精度	参数量	优点	缺点
方法3	DistilBERT (冻结)	逻辑回归	**84.28%**	1.5K	上下文感知，训练极快	需要大模型提取特征（768维）
方法4	DistilBERT (冻结)	Few-shot 原型	**66.57%**	—	无需训练，适应新类别	精度低，不稳定（±4.1%）

关键结论

1. 精度排名

Word2Vec + BiGRU (90.85%) > One-Hot + MLP (85.05%) > DistilBERT + LR (84.28%) > Few-shot (66.57%)

2. 参数量效率

- **DistilBERT + LR** 仅 1,538 个参数，精度达 84.28%（性价比最高）
- **Word2Vec** 比 One-Hot 参数量更少（17.6M vs 21.1M），精度更高（+5.8%）

3. 语义捕获能力

方法	词义相似度	词序	上下文
One-Hot	❌	❌	❌
Word2Vec	✅	✅ (通过BiGRU)	❌
DistilBERT	✅	✅	✅

4. 适用场景

- **One-Hot + MLP**：小词表、快速基线模型
- **Word2Vec + BiGRU**：中等数据量，需要捕获词序信息
- **DistilBERT + LR**：有 GPU，追求精度与效率平衡
- **Few-shot**：只有少量标注样本（如20个），无法训练模型时

In [30]:

```python
# ==================== 5. 展示三个测试样本的预测结果 ====================
print("=" * 60)
print("Prediction results of the first 3 samples in the test set")
print("=" * 60)

sample_texts = results["sample_texts"]
true_labels = results["sample_true_labels"]

# Predictions and probabilities for each method
methods = {
    "Method 1 (One-Hot + MLP)": results["method1_onehot"],
    "Method 2 (Word2Vec + BiGRU)": results["method2_w2v_gru"],
    "Method 3 (DistilBERT + LogReg)": results["method3_distilbert_lr"],
    "Method 4 (Few-shot)": results["method4_fewshot"]
}

for i, (text, true) in enumerate(zip(sample_texts, true_labels)):
    print(f"\nSample {i+1}:")
    print(f"True sentiment: {'Positive' if true == 1 else 'Negative'} (1=Positive, 0=Negative)")
    print(f"Review summary: {text[:150]}...\n")
    for method_name, pred_data in methods.items():
        pred = pred_data["predictions"][i]
        prob = pred_data["probabilities"][i]
        prob_pos = prob[1]  # Probability of positive
        prob_neg = prob[0]
        sentiment = "Positive" if pred == 1 else "Negative"
        print(f"  {method_name}: Prediction = {sentiment} (Probabilities: Positive {prob_pos:.4f}, Negative {prob_neg:.4f})")
    print("-" * 50)
```

```
============================================================
Prediction results of the first 3 samples in the test set
============================================================

Sample 1:
True sentiment: Positive (1=Positive, 0=Negative)
Review summary: i went <UNK> saw this movie last night after being coaxed to by a few friends of mine i'll admit that i was rel
uctant to see it because from what i kn...

  Method 1 (One-Hot + MLP): Prediction = Positive (Probabilities: Positive 0.8426, Negative 0.1574)
  Method 2 (Word2Vec + BiGRU): Prediction = Positive (Probabilities: Positive 0.9073, Negative 0.0927)
  Method 3 (DistilBERT + LogReg): Prediction = Positive (Probabilities: Positive 0.6615, Negative 0.3385)
  Method 4 (Few-shot): Prediction = Negative (Probabilities: Positive 0.4992, Negative 0.5008)
------------------------------------------------------

Sample 2:
True sentiment: Positive (1=Positive, 0=Negative)
Review summary: actor turned director bill paxton follows up his promising debut gothic horror frailty with this family friendl
y sports drama about 1913 u s open wher...

  Method 1 (One-Hot + MLP): Prediction = Positive (Probabilities: Positive 0.9789, Negative 0.0211)
  Method 2 (Word2Vec + BiGRU): Prediction = Positive (Probabilities: Positive 0.9892, Negative 0.0108)
  Method 3 (DistilBERT + LogReg): Prediction = Positive (Probabilities: Positive 0.9520, Negative 0.0480)
  Method 4 (Few-shot): Prediction = Positive (Probabilities: Positive 0.5044, Negative 0.4956)
------------------------------------------------------

Sample 3:
True sentiment: Positive (1=Positive, 0=Negative)
Review summary: as a recreational golfer with some knowledge of sport's history i was pleased with disney's sensitivity to issu
es of class in golf in early twentieth ...

  Method 1 (One-Hot + MLP): Prediction = Positive (Probabilities: Positive 0.9064, Negative 0.0936)
  Method 2 (Word2Vec + BiGRU): Prediction = Positive (Probabilities: Positive 0.9746, Negative 0.0254)
  Method 3 (DistilBERT + LogReg): Prediction = Positive (Probabilities: Positive 0.9220, Negative 0.0780)
  Method 4 (Few-shot): Prediction = Negative (Probabilities: Positive 0.4980, Negative 0.5020)
------------------------------------------------------
```

代码展示

In [ ]:

```
# -*- coding: utf-8 -*-
"""
IMDB 情感分析 - 符合作业要求版
- 方法1: One-Hot 词袋 (unigram) + MLP （分析参数量）
- 方法2: Word2Vec(300d, 全量数据) + BiGRU
- 方法3: DistilBERT 冻结 + 强正则化逻辑回归（全量数据）
- 方法4: Few-shot (20-shot, 10次集成评估)
"""

# 导入操作系统相关功能，用于文件路径和目录操作
import os
# 导入JSON模块，用于读取和写入JSON格式的数据
import json
# 导入随机数模块，用于生成随机数（如随机采样）
import random
# 导入垃圾回收模块，用于手动释放内存
import gc
# 从collections模块导入计数器，用于统计词频
from collections import Counter
# 从pathlib导入Path类，用于面向对象的路径操作
from pathlib import Path

# 导入NumPy库，用于高效的数值计算和数组操作
import numpy as np
# 导入PyTorch深度学习框架
import torch
# 导入PyTorch神经网络模块
import torch.nn as nn
# 导入PyTorch优化器模块
import torch.optim as optim
# 导入PyTorch数据加载相关的工具
from torch.utils.data import DataLoader, Dataset, TensorDataset
# 从PyTorch的RNN工具中导入序列填充函数
from torch.nn.utils.rnn import pad_sequence
# 导入进度条库，用于显示训练进度
from tqdm import tqdm
# 从scikit-learn导入文本特征提取的CountVectorizer，用于构建词袋模型
from sklearn.feature_extraction.text import CountVectorizer
# 从scikit-learn导入标准化器，用于数据标准化
from sklearn.preprocessing import StandardScaler
```

```python
# 导入gensim库的Word2Vec模型，用于训练词向量
from gensim.models import Word2Vec
# 从transformers库导入分词器和预训练模型加载工具
from transformers import AutoTokenizer, AutoModel

# 定义设置随机种子的函数，确保实验结果可复现
def set_seed(seed=42):
    random.seed(seed)  # 设置Python内置随机数的种子
    np.random.seed(seed)  # 设置NumPy随机数的种子
    torch.manual_seed(seed)  # 设置PyTorch CPU随机数的种子
    torch.cuda.manual_seed_all(seed)  # 设置PyTorch所有GPU的随机数种子
    torch.backends.cudnn.deterministic = True  # 设置CuDNN为确定性模式
    torch.backends.cudnn.benchmark = False  # 关闭CuDNN的自动优化，确保可复现

# 调用函数设置随机种子为42
set_seed(42)

# 检测是否有可用的GPU，如果没有则使用CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# 打印当前使用的设备（GPU或CPU）
print(f"Using device: {device}")

# ==================== 路径配置 ====================
# 设置数据目录的路径
DATA_DIR = "/Pytorch_Book_ZhouRUC/dataset"
# 设置IMDB数据文件的NPZ格式路径
NPZ_PATH = os.path.join(DATA_DIR, "imdb.npz")
# 设置词索引JSON文件的路径
JSON_PATH = os.path.join(DATA_DIR, "imdb_word_index.json")
# 设置DistilBERT预训练模型的本地路径
DISTILBERT_PATH = "/mnt/Data/distilbert-base-uncased"

# ==================== 1. 数据加载与解码 ====================
# 定义从NPZ和JSON文件加载IMDB数据的函数
def load_imdb_from_npz(npz_path, json_path):
    # 使用NumPy加载npz文件，允许pickle序列化
    data = np.load(npz_path, allow_pickle=True)
    # 提取训练数据和标签
    x_train, y_train = data['x_train'], data['y_train']
    # 提取测试数据和标签
    x_test, y_test = data['x_test'], data['y_test']
```

```
    # 打开并加载词索引JSON文件
    with open(json_path, 'r') as f:
        word_index = json.load(f)
    # 构建索引到单词的映射字典（反向映射）
    index_to_word = {v: k for k, v in word_index.items()}
    # 手动添加特殊标记的映射
    index_to_word[0] = '<PAD>'  # 填充标记
    index_to_word[1] = '<START>'  # 开始标记
    index_to_word[2] = '<UNK>'  # 未知词标记
    # 打印词汇表大小
    print(f"Vocab size: {len(word_index)}")
    # 打印训练集和测试集的大小
    print(f"Train: {len(x_train)}, Test: {len(x_test)}")
    # 返回所有数据和映射字典
    return (x_train, y_train), (x_test, y_test), word_index, index_to_word

# 定义将编码后的评论解码为原始文本的函数
def decode_review(encoded_review, index_to_word, max_len=None):
    words = []  # 初始化单词列表
    # 遍历编码后的整数序列
    for idx in encoded_review:
        # 跳过填充和开始标记
        if idx in [0, 1]:
            continue
        # 根据索引获取单词，未知词用<UNK>代替
        word = index_to_word.get(idx, '<UNK>')
        words.append(word)  # 将单词加入列表
        # 如果设置了最大长度且已达到，则提前结束
        if max_len and len(words) >= max_len:
            break
    # 用空格连接单词列表，返回完整的评论文本
    return ' '.join(words)

# 调用函数加载全量数据
(x_train, y_train), (x_test, y_test), word_index, index_to_word = load_imdb_from_npz(NPZ_PATH, JSON_PATH)

# 打印解码开始提示
print("Decoding all texts (this may take 2-3 minutes)...")
# 解码所有训练集评论，使用进度条显示进度
train_texts = [decode_review(seq, index_to_word) for seq in tqdm(x_train, desc="Train decode")]
# 解码所有测试集评论，使用进度条显示进度
```

```python
test_texts = [decode_review(seq, index_to_word) for seq in tqdm(x_test, desc="Test decode")]
# 将训练标签转换为Python列表格式
train_labels = y_train.tolist() if hasattr(y_train, 'tolist') else list(y_train)
# 将测试标签转换为Python列表格式
test_labels = y_test.tolist() if hasattr(y_test, 'tolist') else list(y_test)
# 打印解码完成后的数据量信息
print(f"Decoded {len(train_texts)} train, {len(test_texts)} test")

# ==================== 公用分类器 ====================
# 定义多层感知机（MLP）分类器类
class MLPClassifier(nn.Module):
    # 初始化函数：input_dim输入维度，hidden_dims隐藏层维度列表，dropout丢弃率
    def __init__(self, input_dim, hidden_dims=[512, 256], dropout=0.5):
        super().__init__()  # 调用父类初始化
        layers = []  # 初始化层列表
        prev = input_dim  # 当前输入维度设为初始输入维度
        # 遍历每个隐藏层维度
        for h in hidden_dims:
            layers.append(nn.Linear(prev, h))  # 添加全连接层
            layers.append(nn.BatchNorm1d(h))  # 添加批归一化层
            layers.append(nn.ReLU())  # 添加ReLU激活函数
            layers.append(nn.Dropout(dropout))  # 添加Dropout层防止过拟合
            prev = h  # 更新当前维度为这一层的输出维度
        layers.append(nn.Linear(prev, 2))  # 添加最终的输出层（二分类，输出2个logit）
        self.net = nn.Sequential(*layers)  # 将所有层组合成序列网络

    # 前向传播函数
    def forward(self, x):
        return self.net(x)  # 直接通过序列网络计算输出

# 定义逻辑回归分类器类（用于方法3）
class LogisticRegression(nn.Module):
    """用于方法3的强正则化线性分类器"""
    # 初始化函数：input_dim输入维度
    def __init__(self, input_dim):
        super().__init__()  # 调用父类初始化
        self.fc = nn.Linear(input_dim, 2)  # 添加一个线性层，输出2个类别

    # 前向传播函数
    def forward(self, x):
        return self.fc(x)  # 直接通过线性层计算输出
```

```python
# ==================== 方法1: One-Hot 词袋 (BOW) + MLP ====================
# 打印方法1的标题分隔线
print("\n" + "="*60)
print("Method 1: One-Hot Bag-of-Words (unigram) + MLP")
print("="*60)

# 构建词袋one-hot向量器，binary=True表示词出现与否（文档级聚合），max_features限制最大特征数
vectorizer_onehot = CountVectorizer(binary=True, max_features=20000)  # 限制词表大小，否则维度爆炸
# 将训练文本转换为one-hot矩阵并转为float32类型的密集数组
X_train_onehot = vectorizer_onehot.fit_transform(train_texts).toarray().astype(np.float32)
# 将测试文本转换为one-hot矩阵并转为float32类型的密集数组
X_test_onehot = vectorizer_onehot.transform(test_texts).toarray().astype(np.float32)
# 打印one-hot向量的形状信息
print(f"One-hot BOW shape: train {X_train_onehot.shape}, test {X_test_onehot.shape}")
# 打印实际词汇表大小
print(f"Vocabulary size: {len(vectorizer_onehot.vocabulary_)}")

# 初始化标准化器，with_mean=False适合稀疏数据（但这里已转为密集数组）
scaler_onehot = StandardScaler(with_mean=False)  # one-hot 是稀疏的，均值缩放会破坏稀疏性，但这里转为 dense 后可以
# 对训练集one-hot向量进行标准化并记录参数
X_train_onehot = scaler_onehot.fit_transform(X_train_onehot)
# 使用训练集的标准化参数对测试集进行转换
X_test_onehot = scaler_onehot.transform(X_test_onehot)

# 定义NumPy数组数据集类，将NumPy数组包装为PyTorch Dataset
class ArrayDataset(Dataset):
    # 初始化函数：接收特征X和标签y
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)  # 转换为PyTorch张量，数据类型float32
        self.y = torch.tensor(y, dtype=torch.long)  # 转换为PyTorch张量，数据类型long（用于分类）

    # 返回数据集长度
    def __len__(self):
        return len(self.y)

    # 根据索引返回一个样本
    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]  # 返回特征和标签

# 设置批大小为128
```

```
batch_size = 128
# 创建训练数据集对象
train_dataset1 = ArrayDataset(X_train_onehot, train_labels)
# 创建测试数据集对象
test_dataset1 = ArrayDataset(X_test_onehot, test_labels)
# 创建训练数据加载器，启用shuffle打乱数据，使用3个子进程
train_loader1 = DataLoader(train_dataset1, batch_size=batch_size, shuffle=True, num_workers=3)
# 创建测试数据加载器，不打乱顺序
test_loader1 = DataLoader(test_dataset1, batch_size=batch_size, shuffle=False, num_workers=3)

# 创建MLP分类器实例，输入维度为one-hot特征维度，指定隐藏层和dropout率
model1 = MLPClassifier(X_train_onehot.shape[1], hidden_dims=[1024, 512, 256], dropout=0.5).to(device)
# 计算并打印模型的总参数量
print(f"Method1 total params: {sum(p.numel() for p in model1.parameters()):,}")

# 定义交叉熵损失函数
criterion = nn.CrossEntropyLoss()
# 定义AdamW优化器，学习率1e-3，权重衰减1e-4
optimizer1 = optim.AdamW(model1.parameters(), lr=1e-3, weight_decay=1e-4)
# 定义余弦退火学习率调度器，周期为20个epoch
scheduler1 = optim.lr_scheduler.CosineAnnealingLR(optimizer1, T_max=20)

# 设置训练轮数
epochs1 = 20
# 初始化最佳测试准确率
best_acc1 = 0
# 设置早停的耐心值（连续多少次验证不提升就停止）
patience = 3
# 初始化连续未提升的轮数计数器
epochs_no_improve = 0
# 设置最佳模型保存路径
best_model_path1 = "Saving/model1_onehot.pt"

# 开始训练循环
for epoch in range(epochs1):
    model1.train()  # 切换到训练模式
    total_loss, correct, total = 0, 0, 0  # 初始化损失、正确数、总数
    # 遍历训练数据加载器，使用进度条显示
    for X, y in tqdm(train_loader1, desc=f"Method1 Epoch {epoch+1}"):
        X, y = X.to(device), y.to(device)  # 将数据移动到设备（GPU/CPU）
        optimizer1.zero_grad()  # 清空梯度
```

```
        logits = model1(X)  # 前向传播，获取模型输出logits
        loss = criterion(logits, y)  # 计算损失
        loss.backward()  # 反向传播计算梯度
        optimizer1.step()  # 更新模型参数
        total_loss += loss.item() * X.size(0)  # 累积总损失
        pred = logits.argmax(dim=1)  # 获取预测类别（最大logit对应的索引）
        correct += (pred == y).sum().item()  # 累积正确预测数
        total += X.size(0)  # 累积总样本数
    train_acc = correct / total  # 计算训练准确率
    scheduler1.step()  # 更新学习率调度器

    model1.eval()  # 切换到评估模式
    correct, total = 0, 0  # 重置正确数和总数
    with torch.no_grad():  # 禁用梯度计算，节省内存和计算
        # 遍历测试数据加载器
        for X, y in test_loader1:
            X, y = X.to(device), y.to(device)  # 移动到设备
            logits = model1(X)  # 前向传播
            pred = logits.argmax(dim=1)  # 获取预测类别
            correct += (pred == y).sum().item()  # 累积正确数
            total += X.size(0)  # 累积总数
    test_acc = correct / total  # 计算测试准确率
    # 打印当前轮次的训练和测试准确率
    print(f"Epoch {epoch+1}: train_acc={train_acc:.4f}, test_acc={test_acc:.4f}")

    # 如果当前测试准确率超过最佳准确率
    if test_acc > best_acc1:
        best_acc1 = test_acc  # 更新最佳准确率
        torch.save(model1.state_dict(), best_model_path1)  # 保存模型参数
        epochs_no_improve = 0  # 重置未提升计数器
    else:
        epochs_no_improve += 1  # 未提升计数器加1

    # 如果连续patience轮未提升，触发早停
    if epochs_no_improve >= patience:
        print(f"Early stopping triggered after epoch {epoch+1} (no improvement for {patience} epochs).")
        break

# 加载最佳模型参数
model1.load_state_dict(torch.load(best_model_path1))
# 打印方法1的最佳测试准确率
```

```
print(f"Method1 Best Test Acc: {best_acc1*100:.2f}%")

# 选取3个测试样本进行预测展示
sample_indices = [0, 1, 2]  # 前3个测试样本的索引
sample_texts = [test_texts[i] for i in sample_indices]  # 获取对应的文本
sample_labels_true = [test_labels[i] for i in sample_indices]  # 获取对应的真实标签
sample_onehot = vectorizer_onehot.transform(sample_texts).toarray().astype(np.float32)  # 转换为one-hot向量
sample_onehot = scaler_onehot.transform(sample_onehot)  # 标准化处理
sample_tensor1 = torch.tensor(sample_onehot, dtype=torch.float32).to(device)  # 转换为PyTorch张量并移动到设备
model1.eval()  # 切换到评估模式
with torch.no_grad():  # 禁用梯度
    logits1 = model1(sample_tensor1)  # 前向传播
    probs1 = torch.softmax(logits1, dim=1).cpu().numpy()  # 计算概率并转为NumPy数组
    preds1 = logits1.argmax(dim=1).cpu().numpy()  # 获取预测类别

# 释放方法1相关的大变量，释放内存
del X_train_onehot, X_test_onehot, scaler_onehot, vectorizer_onehot, model1
del train_loader1, test_loader1
gc.collect()  # 手动运行垃圾回收
torch.cuda.empty_cache()  # 清空CUDA缓存

# ==================== 方法2: Word2Vec + BiGRU ====================
# 打印方法2的标题分隔线
print("\n" + "="*60)
print("Method 2: Word2Vec (300d, full data) + BiGRU")
print("="*60)

# 从gensim导入回调函数基类
from gensim.models.callbacks import CallbackAny2Vec

# 定义Word2Vec训练过程中的回调类，用于记录训练进度
class EpochLogger(CallbackAny2Vec):
    def __init__(self):
        self.epoch = 0  # 当前轮数计数器
        self.total_epochs = None  # 总轮数

    def on_epoch_end(self, model):
        self.epoch += 1  # 轮数加1
        if self.total_epochs is None:
            self.total_epochs = model.epochs  # 获取总轮数
        # 打印当前训练进度
```

```
        print(f"Word2Vec training: epoch {self.epoch}/{self.total_epochs} completed")

# 打印提示信息
print("Training Word2Vec on all training texts...")
# 将训练文本分词(按空格分割)
tokenized_train = [text.split() for text in train_texts]
# 创建并训练Word2Vec模型
w2v_model = Word2Vec(
    sentences=tokenized_train,  # 输入的分词语料
    vector_size=300,  # 词向量维度
    window=5,  # 上下文窗口大小
    min_count=2,  # 忽略出现次数小于2的词
    workers=8,  # 并行线程数
    sg=0,  # 使用CBOW模型（0=CBOW, 1=Skip-gram），CBOW更快
    epochs=15,  # 训练轮数
    callbacks=[EpochLogger()]  # 添加进度回调
)
# 打印Word2Vec词汇表大小
print(f"Word2Vec vocab size: {len(w2v_model.wv)}")

# 建立词到索引的映射字典
word2idx_w2v = {w: i+3 for i, w in enumerate(w2v_model.wv.index_to_key)}  # 从3开始编号
word2idx_w2v['<PAD>'] = 0  # 填充标记索引为0
word2idx_w2v['<START>'] = 1  # 开始标记索引为1
word2idx_w2v['<UNK>'] = 2  # 未知词标记索引为2
# 建立索引到词的逆向映射
idx_to_word_w2v = {v: k for k, v in word2idx_w2v.items()}

# 定义文本到ID序列的转换函数
def text_to_ids(text, max_len=400):
    words = text.split()  # 分词
    # 将每个词转换为对应的ID，未知词使用2（<UNK>），并截断到最大长度
    ids = [word2idx_w2v.get(w, 2) for w in words][:max_len]
    return ids

# 将所有训练文本转换为ID序列，使用进度条
train_ids = [text_to_ids(t) for t in tqdm(train_texts, desc="Train to ids")]
# 将所有测试文本转换为ID序列，使用进度条
test_ids = [text_to_ids(t) for t in tqdm(test_texts, desc="Test to ids")]

# 获取最大索引值
```

```
max_idx = max(word2idx_w2v.values())
# 初始化嵌入矩阵（全零）
embedding_matrix = np.zeros((max_idx + 1, 300), dtype=np.float32)
# 将Word2Vec训练好的词向量填入嵌入矩阵
for w, i in word2idx_w2v.items():
    if i >= 3:  # 只处理真实词（跳过特殊标记）
        embedding_matrix[i] = w2v_model.wv[w]

# 定义Word2Vec数据集类
class W2VDataset(Dataset):
    def __init__(self, ids, labels):
        self.ids = ids  # ID序列列表
        self.labels = labels  # 标签列表

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        # 返回ID序列张量和标签（标签保持原始类型）
        return torch.tensor(self.ids[idx], dtype=torch.long), self.labels[idx]

# 定义批处理时的collate函数，用于将不同长度的序列填充到相同长度
def collate_w2v(batch, pad_idx=0):
    ids, labels = zip(*batch)  # 解包批次中的ID序列和标签
    # 使用pad_sequence将序列填充到相同长度
    ids_padded = pad_sequence(ids, batch_first=True, padding_value=pad_idx)
    # 返回填充后的ID张量和标签张量
    return ids_padded, torch.tensor(labels, dtype=torch.long)

# 设置批大小
batch_size = 32
# 创建训练数据集
train_dataset2 = W2VDataset(train_ids, train_labels)
# 创建测试数据集
test_dataset2 = W2VDataset(test_ids, test_labels)
# 创建训练数据加载器，使用自定义collate函数
train_loader2 = DataLoader(train_dataset2, batch_size=batch_size, shuffle=True,
                           collate_fn=collate_w2v, num_workers=3)
# 创建测试数据加载器
test_loader2 = DataLoader(test_dataset2, batch_size=batch_size, shuffle=False,
                          collate_fn=collate_w2v, num_workers=3)
```

```python
# 定义BiGRU分类器类
class BiGRUClassifier(nn.Module):
    # 初始化函数：embedding_matrix嵌入矩阵，hidden_dim隐藏层维度，dropout丢弃率
    def __init__(self, embedding_matrix, hidden_dim=256, dropout=0.5):
        super().__init__()
        vocab_size, embed_dim = embedding_matrix.shape  # 获取词汇表大小和嵌入维度
        # 创建嵌入层，padding_idx指定填充标记的索引
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        # 将预训练的词向量权重复制到嵌入层
        self.embedding.weight.data.copy_(torch.from_numpy(embedding_matrix))
        # 冻结嵌入层，不参与训练
        self.embedding.weight.requires_grad = False
        # 定义双向GRU层，num_layers=2表示2层，bidirectional=True表示双向
        self.gru = nn.GRU(embed_dim, hidden_dim, batch_first=True,
                          bidirectional=True, dropout=dropout, num_layers=2)
        # 定义全连接输出层，输入维度是hidden_dim*2（双向拼接）
        self.fc = nn.Linear(hidden_dim*2, 2)
        # 定义Dropout层
        self.dropout = nn.Dropout(dropout)

    # 前向传播函数
    def forward(self, x):
        emb = self.embedding(x)  # 将ID序列转换为词向量序列
        out, _ = self.gru(emb)  # 通过GRU层，只取输出（隐藏状态）
        out = out[:, -1, :]  # 取最后一个时间步的输出
        out = self.dropout(out)  # 应用Dropout
        return self.fc(out)  # 通过全连接层输出分类logits

# 创建BiGRU模型实例
model2 = BiGRUClassifier(embedding_matrix, hidden_dim=256, dropout=0.5).to(device)
# 打印模型参数量
print(f"Method2 params: {sum(p.numel() for p in model2.parameters()):,}")

# 定义AdamW优化器
optimizer2 = optim.AdamW(model2.parameters(), lr=1e-3, weight_decay=1e-4)
# 定义余弦退火学习率调度器
scheduler2 = optim.lr_scheduler.CosineAnnealingLR(optimizer2, T_max=15)

# 设置训练轮数
epochs2 = 15
```

```python
# 初始化最佳准确率
best_acc2 = 0
# 设置早停耐心值
patience = 3
# 初始化未提升计数器
epochs_no_improve = 0
# 设置最佳模型保存路径
best_model_path2 = "Saving/model2_w2v_gru.pt"

# 开始训练循环
for epoch in range(epochs2):
    model2.train()  # 切换到训练模式
    total_loss, correct, total = 0, 0, 0
    # 遍历训练数据加载器
    for x, y in tqdm(train_loader2, desc=f"Method2 Epoch {epoch+1}"):
        x, y = x.to(device), y.to(device)  # 移动到设备
        optimizer2.zero_grad()  # 清空梯度
        logits = model2(x)  # 前向传播
        loss = criterion(logits, y)  # 计算损失
        loss.backward()  # 反向传播
        optimizer2.step()  # 更新参数
        total_loss += loss.item() * x.size(0)  # 累积损失
        pred = logits.argmax(dim=1)  # 获取预测
        correct += (pred == y).sum().item()  # 累积正确数
        total += x.size(0)  # 累积总数
    train_acc = correct / total  # 计算训练准确率
    scheduler2.step()  # 更新学习率

    model2.eval()  # 切换到评估模式
    correct, total = 0, 0
    with torch.no_grad():
        for x, y in test_loader2:
            x, y = x.to(device), y.to(device)
            logits = model2(x)
            pred = logits.argmax(dim=1)
            correct += (pred == y).sum().item()
            total += x.size(0)
    test_acc = correct / total
    print(f"Epoch {epoch+1}: train_acc={train_acc:.4f}, test_acc={test_acc:.4f}")

    # 保存最佳模型和早停判断
```

```
    if test_acc > best_acc2:
        best_acc2 = test_acc
        torch.save(model2.state_dict(), best_model_path2)
        epochs_no_improve = 0
    else:
        epochs_no_improve += 1

    if epochs_no_improve >= patience:
        print(f"Early stopping triggered after epoch {epoch+1} (no improvement for {patience} epochs).")
        break

# 加载最佳模型参数
model2.load_state_dict(torch.load(best_model_path2))
# 打印方法2的最佳测试准确率
print(f"Method2 Best Test Acc: {best_acc2*100:.2f}%")

# 对3个样本进行预测
model2.eval()
sample_ids2 = [text_to_ids(t) for t in sample_texts]  # 将样本文本转换为ID序列
# 填充序列并移动到设备
sample_ids_padded = pad_sequence([torch.tensor(ids) for ids in sample_ids2], batch_first=True).to(device)
with torch.no_grad():
    logits2 = model2(sample_ids_padded)  # 前向传播
    probs2 = torch.softmax(logits2, dim=1).cpu().numpy()  # 计算概率
    preds2 = logits2.argmax(dim=1).cpu().numpy()  # 获取预测类别

# 释放方法2相关变量，释放内存
del train_ids, test_ids, embedding_matrix, w2v_model, model2
del train_loader2, test_loader2
gc.collect()
torch.cuda.empty_cache()

# ==================== 方法3: DistilBERT 冻结 + 逻辑回归（全量数据）====================
print("\n" + "="*60)
print("Method 3: Frozen DistilBERT + Logistic Regression (full data)")
print("="*60)

# 设置缓存的嵌入文件路径
train_emb_cache = "Saving/distilbert_emb_train.npy"
test_emb_cache = "Saving/distilbert_emb_test.npy"
# 检查缓存文件是否存在
```

```python
if os.path.exists(train_emb_cache) and os.path.exists(test_emb_cache):
    # 如果存在，直接加载缓存的嵌入
    train_embs = np.load(train_emb_cache)
    test_embs = np.load(test_emb_cache)
    print("Loaded cached DistilBERT embeddings.")
else:
    # 如果不存在，加载分词器和DistilBERT模型
    tokenizer = AutoTokenizer.from_pretrained(DISTILBERT_PATH)
    model_bert = AutoModel.from_pretrained(DISTILBERT_PATH).to(device)
    model_bert.eval()  # 设置为评估模式，冻结所有参数

    # 定义提取嵌入的函数
    def extract_embeddings(texts, batch_size=64):
        embs = []  # 初始化嵌入列表
        with torch.no_grad():  # 禁用梯度计算，节省内存
            # 分批处理文本
            for i in tqdm(range(0, len(texts), batch_size), desc="Extracting"):
                batch = texts[i:i+batch_size]  # 获取当前批次
                # 使用分词器将文本转换为模型输入格式
                inputs = tokenizer(batch, return_tensors='pt', padding=True,
                                   truncation=True, max_length=256).to(device)
                outputs = model_bert(**inputs)  # 前向传播
                # 获取[CLS]标记的嵌入（第一个token），并移回CPU转为NumPy
                cls_emb = outputs.last_hidden_state[:, 0, :].cpu().numpy()
                embs.append(cls_emb)  # 添加到列表
        return np.concatenate(embs, axis=0)  # 拼接所有批次的嵌入

    print("Extracting training embeddings...")
    train_embs = extract_embeddings(train_texts, batch_size=64)  # 提取训练集嵌入
    np.save(train_emb_cache, train_embs)  # 保存到缓存文件
    print("Extracting test embeddings...")
    test_embs = extract_embeddings(test_texts, batch_size=64)  # 提取测试集嵌入
    np.save(test_emb_cache, test_embs)  # 保存到缓存文件
    del model_bert  # 释放BERT模型，释放显存

# 打印嵌入数组的形状
print(f"Train embeddings shape: {train_embs.shape}, Test: {test_embs.shape}")

# 使用逻辑回归（线性分类器）+ 强正则化
# 创建训练嵌入的数据集
train_emb_dataset = ArrayDataset(train_embs, train_labels)
```

```python
# 创建测试嵌入的数据集
test_emb_dataset = ArrayDataset(test_embs, test_labels)
# 创建训练数据加载器，批大小512
train_emb_loader = DataLoader(train_emb_dataset, batch_size=512, shuffle=True, num_workers=3)
# 创建测试数据加载器
test_emb_loader = DataLoader(test_emb_dataset, batch_size=512, shuffle=False, num_workers=3)

# 创建逻辑回归模型实例
model3 = LogisticRegression(train_embs.shape[1]).to(device)
# 打印模型参数量
print(f"Method3 params: {sum(p.numel() for p in model3.parameters()):,}")

# 定义优化器，使用较高的权重衰减（1e-1）实现强L2正则化
optimizer3 = optim.AdamW(model3.parameters(), lr=1e-3, weight_decay=1e-1)  # 强L2正则
criterion3 = nn.CrossEntropyLoss()  # 交叉熵损失
epochs3 = 20  # 训练轮数
best_acc3 = 0  # 最佳准确率
patience = 3  # 早停耐心值
epochs_no_improve = 0
best_model_path3 = "Saving/model3_distilbert_lr.pt"

# 开始训练循环
for epoch in range(epochs3):
    model3.train()  # 训练模式
    total_loss, correct, total = 0, 0, 0
    for X, y in tqdm(train_emb_loader, desc=f"Method3 Epoch {epoch+1}"):
        X, y = X.to(device), y.to(device)  # 移动到设备
        optimizer3.zero_grad()  # 清空梯度
        logits = model3(X)  # 前向传播
        loss = criterion3(logits, y)  # 计算损失
        loss.backward()  # 反向传播
        optimizer3.step()  # 更新参数
        total_loss += loss.item() * X.size(0)  # 累积损失
        pred = logits.argmax(dim=1)  # 获取预测
        correct += (pred == y).sum().item()  # 累积正确数
        total += X.size(0)  # 累积总数
    train_acc = correct / total  # 训练准确率

    model3.eval()  # 评估模式
    correct, total = 0, 0
    with torch.no_grad():
```

```
        for X, y in test_emb_loader:
            X, y = X.to(device), y.to(device)
            logits = model3(X)
            pred = logits.argmax(dim=1)
            correct += (pred == y).sum().item()
            total += X.size(0)
    test_acc = correct / total
    print(f"Epoch {epoch+1}: train_acc={train_acc:.4f}, test_acc={test_acc:.4f}")

    # 保存最佳模型和早停判断
    if test_acc > best_acc3:
        best_acc3 = test_acc
        torch.save(model3.state_dict(), best_model_path3)
        epochs_no_improve = 0
    else:
        epochs_no_improve += 1

    if epochs_no_improve >= patience:
        print(f"Early stopping triggered after epoch {epoch+1} (no improvement for {patience} epochs).")
        break

# 加载最佳模型参数
model3.load_state_dict(torch.load(best_model_path3))
print(f"Method3 Best Test Acc: {best_acc3*100:.2f}%")

# 对3个样本进行预测
tokenizer = AutoTokenizer.from_pretrained(DISTILBERT_PATH)  # 重新加载分词器
model_bert = AutoModel.from_pretrained(DISTILBERT_PATH).to(device)  # 重新加载BERT模型
model_bert.eval()  # 评估模式
sample_embs = []  # 存储样本嵌入
with torch.no_grad():
    for t in sample_texts:  # 遍历样本文本
        inputs = tokenizer(t, return_tensors='pt', truncation=True, max_length=256).to(device)
        outputs = model_bert(**inputs)  # 前向传播
        emb = outputs.last_hidden_state[:, 0, :].cpu().numpy()  # 提取[CLS]嵌入
        sample_embs.append(emb[0])  # 添加到列表
sample_embs = np.array(sample_embs)  # 转为NumPy数组
sample_tensor3 = torch.tensor(sample_embs, dtype=torch.float32).to(device)  # 转为张量
model3.eval()  # 评估模式
with torch.no_grad():
    logits3 = model3(sample_tensor3)  # 前向传播
```

```
    probs3 = torch.softmax(logits3, dim=1).cpu().numpy()  # 计算概率
    preds3 = logits3.argmax(dim=1).cpu().numpy()  # 获取预测类别
del model_bert  # 释放BERT模型，释放显存

# ==================== 方法4: Few-shot (20-shot, 10次集成) ====================
print("\n" + "="*60)
print("Method 4: Few-shot (20-shot, 10 trials)")
print("="*60)

# 使用相同的DistilBERT模型提取嵌入
model_fs = AutoModel.from_pretrained(DISTILBERT_PATH).to(device)  # 加载模型
model_fs.eval()  # 评估模式

# 定义获取单个文本嵌入的函数
def get_embedding(text):
    inputs = tokenizer(text, return_tensors='pt', truncation=True, max_length=256).to(device)
    with torch.no_grad():
        outputs = model_fs(**inputs)  # 前向传播
        # 返回[CLS]标记的嵌入，并移回CPU转为NumPy数组
        return outputs.last_hidden_state[:, 0, :].cpu().numpy()[0]

# 定义few-shot预测函数（基于原型网络）
def few_shot_predict(support_texts, support_labels, query_texts):
    # 计算每个类别的原型（平均嵌入）
    prototypes = {}
    for label in [0, 1]:  # 对两个类别分别计算
        # 收集当前类别所有支持样本的嵌入
        embs = [get_embedding(t) for t, l in zip(support_texts, support_labels) if l == label]
        # 计算平均嵌入作为原型，如果该类别没有样本则使用零向量
        prototypes[label] = np.mean(embs, axis=0) if embs else np.zeros(768)
    # 对查询样本进行预测
    preds, probs = [], []
    for text in query_texts:  # 遍历每个查询文本
        emb = get_embedding(text)  # 获取查询样本的嵌入
        # 计算与类别原型的余弦相似度（添加小值防止除零）
        sim0 = np.dot(emb, prototypes[0]) / (np.linalg.norm(emb)*np.linalg.norm(prototypes[0])+1e-8)
        sim1 = np.dot(emb, prototypes[1]) / (np.linalg.norm(emb)*np.linalg.norm(prototypes[1])+1e-8)
        # 使用相似度计算概率（softmax）
        exp0, exp1 = np.exp(sim0), np.exp(sim1)
        prob0 = exp0/(exp0+exp1)  # 类别0的概率
        pred = 0 if sim0 > sim1 else 1  # 根据相似度大小决定预测类别
```

```
        preds.append(pred)  # 添加预测
        probs.append([prob0, 1-prob0])  # 添加概率
    return np.array(preds), np.array(probs)  # 返回预测和概率

# 评估few-shot性能
n_trials = 10  # 试验次数
k_shot = 20  # 每类20个样本
query_size = 2000  # 每次评估查询2000个测试样本
accs = []  # 存储每次试验的准确率
# 进行多次试验
for trial in range(n_trials):
    # 获取正例（标签1）和负例（标签0）的索引
    pos_idx = [i for i,l in enumerate(train_labels) if l==1]
    neg_idx = [i for i,l in enumerate(train_labels) if l==0]
    # 随机采样支持样本（每类k_shot个）
    support_pos = random.sample(pos_idx, k_shot)
    support_neg = random.sample(neg_idx, k_shot)
    # 获取支持集的文本和标签
    support_texts_fs = [train_texts[i] for i in support_pos+support_neg]
    support_labels_fs = [1]*k_shot + [0]*k_shot
    # 随机采样查询样本（从测试集中）
    query_idx = random.sample(range(len(test_texts)), query_size)
    query_texts_fs = [test_texts[i] for i in query_idx]
    query_labels_fs = [test_labels[i] for i in query_idx]
    # 使用few-shot预测
    preds, _ = few_shot_predict(support_texts_fs, support_labels_fs, query_texts_fs)
    acc = np.mean(preds == query_labels_fs)  # 计算准确率
    accs.append(acc)  # 记录准确率
    print(f"Trial {trial+1}: acc={acc*100:.2f}%")
# 计算平均准确率和标准差
mean_acc4 = np.mean(accs)
std_acc4 = np.std(accs)
print(f"Few-shot (20-shot, {n_trials} trials) Test Acc: {mean_acc4*100:.2f}% ± {std_acc4*100:.2f}%")

# 对3个样本进行预测（使用最后一次试验的支持集）
preds_fewshot_sample, probs_fewshot_sample = few_shot_predict(support_texts_fs, support_labels_fs, sample_texts)
del model_fs  # 释放模型

# ==================== 保存结果 ====================
# 创建保存目录（如果不存在）
os.makedirs("Saving", exist_ok=True)
```

```
os.makedirs("Output", exist_ok=True)

# 构建结果字典
results = {
    "sample_texts": sample_texts,  # 样本文本
    "sample_true_labels": sample_labels_true,  # 样本真实标签
    "method1_onehot": {"predictions": preds1.tolist(), "probabilities": probs1.tolist()},  # 方法1结果
    "method2_w2v_gru": {"predictions": preds2.tolist(), "probabilities": probs2.tolist()},  # 方法2结果
    "method3_distilbert_lr": {"predictions": preds3.tolist(), "probabilities": probs3.tolist()},  # 方法3结果
    "method4_fewshot": {"predictions": preds_fewshot_sample.tolist(), "probabilities": probs_fewshot_sample.tolist()},  # 方法
    "test_accuracies": {  # 各种方法的测试准确率
        "method1": float(best_acc1),
        "method2": float(best_acc2),
        "method3": float(best_acc3),
        "method4": float(mean_acc4)
    }
}
# 将结果保存为JSON文件
with open("Output/results_final.json", "w") as f:
    json.dump(results, f, indent=2)

# 构建对比总结字符串
comparison = f"""
========================================
FINAL RESULTS (One-Hot, Word2Vec, DistilBERT, Few-shot)
========================================
Method 1 (One-Hot Bag-of-Words + MLP)          : {best_acc1*100:.2f}%
Method 2 (Word2Vec + BiGRU)                    : {best_acc2*100:.2f}%
Method 3 (Frozen DistilBERT + LogReg)          : {best_acc3*100:.2f}%
Method 4 (Few-shot, 20-shot, 10 trials)        : {mean_acc4*100:.2f}% (±{std_acc4*100:.2f}%)

Observations:
- One-hot + MLP performs reasonably but suffers from high dimensionality and lack of semantic information.
- Word2Vec + BiGRU captures word order and semantics, achieving similar accuracy with far fewer parameters.
- Frozen DistilBERT with strong regularization gives the best accuracy due to rich contextual embeddings.
- Few-shot works without training but is less stable and requires a good support set.
"""
# 将对比结果保存为文本文件
with open("Output/comparison_final.txt", "w") as f:
    f.write(comparison)
# 打印对比结果到控制台
```

```
print(comparison)
# 打印保存位置提示
print("\nAll results saved to Output/ and models to Saving/")
```

In [29]:

```
with open("run.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

```
Using device: cuda
Vocab size: 88584
Train: 25000, Test: 25000
Decoding all texts (this may take 2-3 minutes)...
Train decode: 100%|██████████| 25000/25000 [00:02<00:00, 12387.17it/s]
Test decode: 100%|██████████| 25000/25000 [00:01<00:00, 12860.09it/s]
Decoded 25000 train, 25000 test

============================================================
Method 1: One-Hot Bag-of-Words (unigram) + MLP
============================================================
One-hot BOW shape: train (25000, 20000), test (25000, 20000)
Vocabulary size: 20000

Method1 total params: 21,141,250
Method1 Epoch 1: 100%|██████████| 196/196 [00:06<00:00, 31.52it/s]
Epoch 1: train_acc=0.8362, test_acc=0.8505
Method1 Epoch 2: 100%|██████████| 196/196 [00:05<00:00, 33.57it/s]
Epoch 2: train_acc=0.9573, test_acc=0.8352
Method1 Epoch 3: 100%|██████████| 196/196 [00:05<00:00, 34.45it/s]
Epoch 3: train_acc=0.9844, test_acc=0.8300
Method1 Epoch 4: 100%|██████████| 196/196 [00:05<00:00, 34.31it/s]
Epoch 4: train_acc=0.9875, test_acc=0.8299
Early stopping triggered after epoch 4 (no improvement for 3 epochs).
Method1 Best Test Acc: 85.05%

============================================================
Method 2: Word2Vec (300d, full data) + BiGRU
============================================================
Training Word2Vec on all training texts...
Word2Vec training: epoch 1/15 completed
Word2Vec training: epoch 2/15 completed
Word2Vec training: epoch 3/15 completed
Word2Vec training: epoch 4/15 completed
Word2Vec training: epoch 5/15 completed
Word2Vec training: epoch 6/15 completed
Word2Vec training: epoch 7/15 completed
Word2Vec training: epoch 8/15 completed
Word2Vec training: epoch 9/15 completed
Word2Vec training: epoch 10/15 completed
Word2Vec training: epoch 11/15 completed
```

```
Word2Vec training: epoch 12/15 completed
Word2Vec training: epoch 13/15 completed
Word2Vec training: epoch 14/15 completed
Word2Vec training: epoch 15/15 completed
Word2Vec vocab size: 51955
Train to ids: 100%|██████████| 25000/25000 [00:01<00:00, 14093.89it/s]
Test to ids: 100%|██████████| 25000/25000 [00:01<00:00, 14735.45it/s]
Method2 params: 17,628,234
Method2 Epoch 1: 100%|██████████| 782/782 [01:23<00:00,  9.40it/s]
Epoch 1: train_acc=0.5501, test_acc=0.8114
Method2 Epoch 2: 100%|██████████| 782/782 [01:23<00:00,  9.38it/s]
Epoch 2: train_acc=0.8678, test_acc=0.9031
Method2 Epoch 3: 100%|██████████| 782/782 [01:23<00:00,  9.38it/s]
Epoch 3: train_acc=0.9120, test_acc=0.9085
Method2 Epoch 4: 100%|██████████| 782/782 [01:23<00:00,  9.38it/s]
Epoch 4: train_acc=0.9344, test_acc=0.9066
Method2 Epoch 5: 100%|██████████| 782/782 [01:23<00:00,  9.40it/s]
Epoch 5: train_acc=0.9554, test_acc=0.9078
Method2 Epoch 6: 100%|██████████| 782/782 [01:23<00:00,  9.38it/s]
Epoch 6: train_acc=0.9720, test_acc=0.9044
Early stopping triggered after epoch 6 (no improvement for 3 epochs).
Method2 Best Test Acc: 90.85%

============================================================
Method 3: Frozen DistilBERT + Logistic Regression (full data)
============================================================
Some weights of the model checkpoint at /mnt/Data/distilbert-base-uncased were not used when initializing DistilBertModel: ['vo
cab_layer_norm.bias', 'vocab_layer_norm.weight', 'vocab_transform.bias', 'vocab_transform.weight', 'vocab_projector.weight', 'v
ocab_projector.bias']
- This IS expected if you are initializing DistilBertModel from the checkpoint of a model trained on another task or with anoth
er architecture (e.g. initializing a BertForSequenceClassification model from a BertForPreTraining model).
- This IS NOT expected if you are initializing DistilBertModel from the checkpoint of a model that you expect to be exactly ide
ntical (initializing a BertForSequenceClassification model from a BertForSequenceClassification model).
Extracting training embeddings...
Extracting: 100%|██████████| 391/391 [02:42<00:00,  2.41it/s]
Extracting test embeddings...
Extracting: 100%|██████████| 391/391 [02:41<00:00,  2.42it/s]
Train embeddings shape: (25000, 768), Test: (25000, 768)
Method3 params: 1,538
Method3 Epoch 1: 100%|██████████| 49/49 [00:01<00:00, 29.77it/s]
Epoch 1: train_acc=0.7321, test_acc=0.7892
```

```
Method3 Epoch 2: 100%|██████████| 49/49 [00:01<00:00, 31.44it/s]
Epoch 2: train_acc=0.8065, test_acc=0.8122
Method3 Epoch 3: 100%|██████████| 49/49 [00:01<00:00, 32.27it/s]
Epoch 3: train_acc=0.8206, test_acc=0.8117
Method3 Epoch 4: 100%|██████████| 49/49 [00:01<00:00, 29.97it/s]
Epoch 4: train_acc=0.8246, test_acc=0.8224
Method3 Epoch 5: 100%|██████████| 49/49 [00:01<00:00, 32.92it/s]
Epoch 5: train_acc=0.8292, test_acc=0.8241
Method3 Epoch 6: 100%|██████████| 49/49 [00:01<00:00, 35.13it/s]
Epoch 6: train_acc=0.8338, test_acc=0.8282
Method3 Epoch 7: 100%|██████████| 49/49 [00:01<00:00, 35.34it/s]
Epoch 7: train_acc=0.8361, test_acc=0.8298
Method3 Epoch 8: 100%|██████████| 49/49 [00:01<00:00, 33.33it/s]
Epoch 8: train_acc=0.8374, test_acc=0.8320
Method3 Epoch 9: 100%|██████████| 49/49 [00:01<00:00, 33.74it/s]
Epoch 9: train_acc=0.8388, test_acc=0.8336
Method3 Epoch 10: 100%|██████████| 49/49 [00:01<00:00, 30.98it/s]
Epoch 10: train_acc=0.8391, test_acc=0.8350
Method3 Epoch 11: 100%|██████████| 49/49 [00:01<00:00, 32.49it/s]
Epoch 11: train_acc=0.8416, test_acc=0.8365
Method3 Epoch 12: 100%|██████████| 49/49 [00:01<00:00, 32.49it/s]
Epoch 12: train_acc=0.8432, test_acc=0.8378
Method3 Epoch 13: 100%|██████████| 49/49 [00:01<00:00, 28.84it/s]
Epoch 13: train_acc=0.8440, test_acc=0.8379
Method3 Epoch 14: 100%|██████████| 49/49 [00:01<00:00, 32.83it/s]
Epoch 14: train_acc=0.8450, test_acc=0.8367
Method3 Epoch 15: 100%|██████████| 49/49 [00:01<00:00, 34.23it/s]
Epoch 15: train_acc=0.8453, test_acc=0.8404
Method3 Epoch 16: 100%|██████████| 49/49 [00:01<00:00, 35.92it/s]
Epoch 16: train_acc=0.8462, test_acc=0.8416
Method3 Epoch 17: 100%|██████████| 49/49 [00:01<00:00, 34.81it/s]
Epoch 17: train_acc=0.8455, test_acc=0.8421
Method3 Epoch 18: 100%|██████████| 49/49 [00:01<00:00, 37.59it/s]
Epoch 18: train_acc=0.8464, test_acc=0.8428
Method3 Epoch 19: 100%|██████████| 49/49 [00:01<00:00, 32.17it/s]
Epoch 19: train_acc=0.8478, test_acc=0.8413
Method3 Epoch 20: 100%|██████████| 49/49 [00:01<00:00, 35.90it/s]
Epoch 20: train_acc=0.8479, test_acc=0.8428
Method3 Best Test Acc: 84.28%
Some weights of the model checkpoint at /mnt/Data/distilbert-base-uncased were not used when initializing DistilBertModel: ['vo
cab_layer_norm.bias', 'vocab_layer_norm.weight', 'vocab_transform.bias', 'vocab_transform.weight', 'vocab_projector.weight', 'v
```

```
ocab_projector.bias']
- This IS expected if you are initializing DistilBertModel from the checkpoint of a model trained on another task or with anoth
er architecture (e.g. initializing a BertForSequenceClassification model from a BertForPreTraining model).
- This IS NOT expected if you are initializing DistilBertModel from the checkpoint of a model that you expect to be exactly ide
ntical (initializing a BertForSequenceClassification model from a BertForSequenceClassification model).

============================================================
Method 4: Few-shot (20-shot, 10 trials)
============================================================
Some weights of the model checkpoint at /mnt/Data/distilbert-base-uncased were not used when initializing DistilBertModel: ['vo
cab_layer_norm.bias', 'vocab_layer_norm.weight', 'vocab_transform.bias', 'vocab_transform.weight', 'vocab_projector.weight', 'v
ocab_projector.bias']
- This IS expected if you are initializing DistilBertModel from the checkpoint of a model trained on another task or with anoth
er architecture (e.g. initializing a BertForSequenceClassification model from a BertForPreTraining model).
- This IS NOT expected if you are initializing DistilBertModel from the checkpoint of a model that you expect to be exactly ide
ntical (initializing a BertForSequenceClassification model from a BertForSequenceClassification model).
Trial 1: acc=69.85%
Trial 2: acc=67.65%
Trial 3: acc=67.25%
Trial 4: acc=72.95%
Trial 5: acc=68.20%
Trial 6: acc=71.45%
Trial 7: acc=63.15%
Trial 8: acc=62.85%
Trial 9: acc=62.65%
Trial 10: acc=59.65%
Few-shot (20-shot, 10 trials) Test Acc: 66.57% ± 4.10%

========================================
FINAL OPTIMIZED RESULTS (DistilBERT only)
========================================
Method 1 (One-Hot Bag-of-Words + MLP)       : 85.05%
Method 2 (Word2Vec + BiGRU)                : 90.85%
Method 3 (Frozen DistilBERT + LogReg)      : 84.28%
Method 4 (Few-shot, 20-shot, 10 trials)    : 66.57% (±4.10%)

Observations:
- One-hot + MLP performs reasonably but suffers from high dimensionality and lack of semantic information.
- Word2Vec + BiGRU captures word order and semantics, achieving similar accuracy with far fewer parameters.
- Frozen DistilBERT with strong regularization gives the best accuracy due to rich contextual embeddings.
- Few-shot works without training but is less stable and requires a good support set.
```

```
All results saved to Output/ and models to Saving/
```